INFORME DE LABORATORIO

INFORMACIÓN BÁSICA

ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TÍTULO DE LA PRÁCTICA:	*Entorno de Ejecución Node.js, Gestión de Paquetes con npm y Creación de Servidor HTTP con Módulos Nativos*				
NÚMERO DE PRÁCTICA:	*5*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*2*
FECHA DE PRESENTACIÓN	*19/10/2026*	**HORA DE PRESENTACIÓN**	*23:59 p.m.*		
INTEGRANTE (s): *Jhosue Joaquin Aguilar Pacheco* *Francisco Antonio Joan Mamani Paucar* *Pacheco Huarca Rodrigo Luis* *Machaca Cotrado Mark Antony*				**NOTA (0-20)**	*Nota colocada por el docente*
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS

I. EJERCICIOS RESUELTOS:

Servidor HTTP básico resuelto utilizando módulos nativos:

```
// server.js
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = 3000;

const server = http.createServer((req, res) => {
    console.log(`Petición recibida: ${req.method} ${req.url}`);

    if (req.url === '/' && req.method === 'GET') {
        const filePath = path.join(__dirname, 'public', 'index.html');
        fs.readFile(filePath, (err, content) => {
            if (err) {
                res.writeHead(500, { 'Content-Type': 'text/plain' });
                res.end('Error interno del servidor');
            } else {
                res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8'
});
```

```
                res.end(content);
            }
        });
    } else if (req.url === '/api/status' && req.method === 'GET') {
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ status: 'OK', uptime: process.uptime() }));
    } else {
        res.writeHead(404, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ message: 'Recurso no encontrado' }));
    }
});

server.listen(PORT, () => {
    console.log(`Servidor escuchando en http://localhost:${PORT}`);
});
```

II. PROBLEMAS PROPUESTOS RESUELTOS:

Caso Práctico: Servidor Backend Estático y API Básica de Consultas Desarrollar una aplicación en Node.js desde cero implementando los siguientes requerimientos:

1. **Configuración de Proyecto npm:**

○ Inicializar el proyecto ejecutando npm init -y.

○ Configurar scripts en el package.json: "start": "node server.js" y "dev": "node --watch server.js".

○ Crear una estructura modular de carpetas: /public (archivos estáticos HTML/CSS), /data (archivo JSON con datos) y server.js.

2. **Construcción del Servidor HTTP Nactivo:**

○ Instanciar el servidor mediante http.createServer escuchando en el puerto 3000.

○ Servir archivos estáticos desde la carpeta /public asíncronamente con fs.readFile infiriendo correctamente el Content-Type (ej. text/html, text/css).

3. **Implementación de Endpoints Basados en Rutas:**

○ Ruta GET /api/estudiantes: Leer el archivo /data/estudiantes.json de forma asíncrona y retornar la lista de estudiantes en formato JSON con código 200 OK.

○ Ruta POST /api/estudiantes: Capturar los chunks de datos del body de la petición (evento req.on('data') y req.on('end')), parsear el objeto, agregarlo al archivo JSON de persistencia local y retornar el registro guardado con código 201 Created.

○ Manejo de rutas no existentes retornando un JSON con código 404 Not Found.

package.json

```
{
  "name": "idweb2",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/f-mamanip/IDWeb2.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/f-mamanip/IDWeb2/issues"
  },
  "homepage": "https://github.com/f-mamanip/IDWeb2#readme"
}
```

INDEX.HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Gestión de Estudiantes</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
    <h1>Bienvenido al Servidor Estático y API</h1>
    <p>El servidor Node.js está funcionando correctamente.</p>
</body>
</html>
```

estudiante.json

```
  {
    "id": 1,
    "nombre": "Ana Torres",
    "carrera": "Ingeniería de Sistemas"
  },
  {
    "nombre": "Carlos Mendoza",
    "carrera": "Ingeniería de Sistemas",
    "id": 2
  }
]
```

server.js

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = 3000;

const server = http.createServer((req, res) => {
    const parsedUrl = new URL(req.url, `http://${req.headers.host}`);
    const pathname = parsedUrl.pathname;

    // 1. ENDPOINT GET /api/estudiantes
    if (req.method === 'GET' && pathname === '/api/estudiantes') {
        const filePath = path.join(__dirname, 'data', 'estudiantes.json');
        fs.readFile(filePath, 'utf8', (err, data) => {
            if (err) {
                res.writeHead(500, { 'Content-Type': 'application/json' });
                res.end(JSON.stringify({ error: 'Error al leer los datos'
}));
                return;
```

```
            }
            res.writeHead(200, { 'Content-Type': 'application/json' });
            res.end(data);
        });
        return;
    }

    // 2. ENDPOINT POST /api/estudiantes
    if (req.method === 'POST' && pathname === '/api/estudiantes') {
        let body = '';

        // Capturar los chunks de datos del body
        req.on('data', chunk => {
            body += chunk.toString();
        });

        req.on('end', () => {
            try {
                const nuevoEstudiante = JSON.parse(body);
                const filePath = path.join(__dirname, 'data',
'estudiantes.json');

                fs.readFile(filePath, 'utf8', (err, data) => {
                    let estudiantes = [];
                    if (!err && data) {
                        estudiantes = JSON.parse(data);
                    }

                    // Asignar ID autoincremental simple
                    nuevoEstudiante.id = estudiantes.length > 0 ?
estudiantes[estudiantes.length - 1].id + 1 : 1;
                    estudiantes.push(nuevoEstudiante);

                    // Guardar en el archivo JSON de persistencia local
                    fs.writeFile(filePath, JSON.stringify(estudiantes, null,
2), (err) => {
                        if (err) {
```

```
                            res.writeHead(500, { 'Content-Type':
'application/json' });
                            res.end(JSON.stringify({ error: 'Error al guardar
el registro' }));
                            return;
                        }
                        res.writeHead(201, { 'Content-Type':
'application/json' });
                        res.end(JSON.stringify(nuevoEstudiante));
                    });
                });
            } catch (error) {
                res.writeHead(400, { 'Content-Type': 'application/json' });
                res.end(JSON.stringify({ error: 'Formato JSON inválido' }));
            }
        });
        return;
    }

    // 3. SERVIR ARCHIVOS ESTÁTICOS DESDE /public
    let filePath = path.join(__dirname, 'public', pathname === '/' ?
'index.html' : pathname);
    const extname = path.extname(filePath);
    let contentType = 'text/html';

    switch (extname) {
        case '.css':
            contentType = 'text/css';
            break;
        case '.js':
            contentType = 'text/javascript';
            break;
        case '.json':
            contentType = 'application/json';
            break;
        case '.png':
            contentType = 'image/png';
            break;
```

```
        case '.jpg':
            contentType = 'image/jpg';
            break;
    }

    fs.readFile(filePath, (err, content) => {
        if (err) {
            if (err.code === 'ENOENT') {
                // Ruta no existente -> 404 Not Found
                res.writeHead(404, { 'Content-Type': 'application/json' });
                res.end(JSON.stringify({ error: 'Ruta no encontrada', status:
404 }));
            } else {
                res.writeHead(500, { 'Content-Type': 'text/plain' });
                res.end(`Error interno del servidor: ${err.code}`);
            }
        } else {
            res.writeHead(200, { 'Content-Type': contentType });
            res.end(content, 'utf-8');
        }
    });
});

server.listen(PORT, () => {
    console.log(`Servidor escuchando en http://localhost:${PORT}`);
});
```

Link GitHub
https://github.com/f-mamanip/IDWeb2

III. CUESTIONARIO:

1. **Explique la arquitectura orientada a eventos y el modelo de I/O No Bloqueante (Non-blocking I/O) de Node.js. ¿En qué se diferencia de servidores multihilo tradicionales como Apache Tomcat?**

- Arquitectura orientada a eventos:

El programa Node.js funciona con eventos y un evento es un aviso de que algo importante pasó dentro del programa. Cuando llega la sugerencia de un usuario o al terminar de leer un archivo o cuando responde la base de datos. En cada aviso el programa tiene una función llamada callback que se activa cuando llega el aviso, es una tarea que queda pendiente para hacerla después.

Se llama arquitectura orientada a eventos porque el programa no sigue una lista consecutiva fija de pasos de principio a fin. Se queda en reposo y responde en cuanto llega el aviso. Para lograrlo Node.js usa varias piezas internas llamadas V8 y call stack y callback queue y libuv y thread pool, por ejemplo el V8 ejecuta el código JavaScript, el call stack lleva el orden de lo que se está ejecutando en ese momento, la callback queue guarda los avisos que ya están listos y que esperan su turno, la libuv y el thread pool le piden al sistema operativo las tareas lentas como leer archivo, la más importante es el event loop, el ciclo que revisa en todo el tiempo si hay avisos en espera, los ejecuta y se comunica con las demás. La mayoría de estas piezas trabajan en segundo plano y el código siempre corre con linealidad de acuerdo a sus tareas.

- I/O no bloqueante:

  El I/O significa entrada y salida, son operaciones como leer un archivo o consultar una base de datos o llamar a otra API. Son lentas ya que dependen de algo fuera del programa. Que sea no bloqueante significa que el programa Node.js no espera a que terminen. Cuando necesita datos de la base de datos pide y deja un callback al responder y mientras tanto atiende a otros usuarios. Cuando la base de datos responde el callback pasa a la callback queue y el event loop lo toma y entrega la respuesta. Se llama programación asíncrona que significa hacer una tarea sin detener todo lo demás.

- Diferencias:

  Tomcat en su modelo tradicional usa un una línea de trabajo del programa por cada petición. Cada usuario que llega recibe su propio hilo y si ese hilo espera a la base de datos queda bloqueado sin hacer nada. Esto tiene dos costos que son la memoria porque cada hilo gasta cerca de 1 MB y el cambio de contexto que ocurre cuando el procesador pierde tiempo saltando entre miles de hilos. Node.js usa un solo hilo que nunca se queda esperando y por eso atiende a miles de conexiones con poca memoria. Su punto débil son las tareas que exigen mucho cálculo como procesar imágenes o encriptar datos porque ocupan el único hilo y todos los demás usuarios quedan esperando. Ahí Tomcat funciona mejor porque sus otros hilos siguen trabajando en distintos núcleos del procesador.

2. ¿Qué función cumple la carpeta node_modules y por qué nunca debe incluirse en el control de versiones (Git)? ¿Qué archivo controla exactamente el árbol de dependencias?

- Función de node_modules:

  La carpeta node_modules es donde npm descarga e instala todas las librerías que usa el proyecto. Estas librerías se llaman dependencias y también se instalan las dependencias de

esas dependencias. Node.js busca el código cuando el programa pide una librería como Express.

- Por qué no se incluye en Git:

  Pesa muchísimo y puede tener cientos de MB y miles de archivos y eso vuelve lenta cada subida y cada descarga. Se puede volver a generar en cualquier momento con el comando npm install y guardarla sería repetir información. Algunas librerías se compilan para un sistema operativo específico y lo que se instaló en un computador puede fallar en otro. Cada cambio genera miles de diferencias que nadie puede revisar. Para evitarlo se escribe node_modules dentro del archivo .gitignore que es la lista de cosas que Git debe ignorar.

- Archivo que controla el árbol de dependencias:

  Es el package-lock.json. Este archivo guarda la versión exacta de cada librería instalada incluidas las que llegan de forma indirecta y así todos los integrantes del equipo y el servidor instalan lo mismo. El package.json solo dice qué librerías quiere el proyecto y con qué rango de versiones. Los dos archivos sí se suben a Git.

3. ¿Cuál es la diferencia entre utilizar métodos síncronos de File System (fs.readFileSync) frente a métodos asíncronos (fs.readFile) en un servidor de producción?

- Método síncrono fs.readFileSync:

  Al ser síncrono el programa se queda en espera hasta terminar de leer todo archivo. Mientras que el servidor no puede atender a el usuario ni a los que no tienen relación con ese archivo y así se bloquea el event loop.

- Método asíncrono fs.readFile:

  Pide la lectura al sistema y sigue atendiendo a otros usuarios al mismo tiempo. Cuando el archivo está listo se activa el callback y continúa con la tarea y sin bloquear el event loop.

- Diferencias:

  En la producción llegan muchos usuarios al mismo tiempo y eso es importante. Si 100 usuarios piden algo y cada lectura síncrona se demora 50 ms el último usuario tiene que esperar 5 segundos. Y si es asíncrono se atienden todos casi a la vez. Por eso en la producción se usa fs.readFile y la versión síncrona solo es aceptable al arrancar la aplicación para cargar configuración cuando no hay usuarios esperando.

CONCLUSIONES

En el laboratorio se construyó un servidor HTTP funcional usando únicamente los módulos nativos de Node.js (http, fs y path), lo que permitió comprender cómo se interpretan las rutas y los métodos de cada petición, y cómo se definen manualmente los códigos de estado y las cabeceras Content-Type. Se comprobó que el modelo orientado a eventos y de I/O no bloqueante permite atender muchas conexiones con un solo hilo, siempre que se usen métodos asíncronos como fs.readFile y fs.writeFile en lugar de sus versiones síncronas. Se implementó una API con los endpoints GET y POST para estudiantes, la lectura del cuerpo por chunks, la persistencia en un archivo JSON y el manejo de errores con respuestas 400, 404 y 500. Asimismo, la configuración con npm (package.json, scripts start y dev) y la organización por carpetas evidenciaron la importancia de mantener proyectos ordenados y reproducibles. En conjunto, la práctica sentó las bases para trabajar posteriormente con frameworks como Express, entendiendo cómo funciona el protocolo HTTP por debajo.

METODOLOGÍA DE TRABAJO

El grupo trabajó de forma colaborativa y con avance continuo, repartiendo las tareas entre los integrantes y revisando en conjunto los resultados de cada etapa. Primero se leyó la guía de laboratorio para identificar los requerimientos y se creó el repositorio en GitHub para compartir el código. Luego se inicializó el proyecto con npm init -y, se configuraron los scripts start y dev, y se organizó la estructura de carpetas (/public, /data y server.js). A continuación se implementó el servidor HTTP con los módulos nativos, desarrollando por partes los endpoints GET y POST de /api/estudiantes, el servicio de archivos estáticos y el manejo de rutas inexistentes. Cada funcionalidad se probó con Postman o curl y el navegador, corrigiendo errores sobre la marcha. En los momentos de dificultad o duda se recurrió al apoyo de herramientas de inteligencia artificial como guía para aclarar conceptos y resolver problemas puntuales, contrastando siempre sus sugerencias con la documentación oficial de Node.js. Finalmente, se respondió el cuestionario teórico con base en lo aprendido y se redactó el informe con los resultados y conclusiones.

REFERENCIAS Y BIBLIOGRAFÍA

- *Node.js Docs. (2025). Node.js v22.x API Reference Documentation: HTTP & FS Modules. Nodejs.org.*
- *Cantelon, M., Harter, M., Holowaychuk, T., & Rajlich, N. (2017). Node.js in Action (2nd ed.). Manning Publications.*
- *Young, A. et al. (2021). Node.js Design Patterns (3rd ed.). Packt Publishing.*
- *https://www.youtube.com/watch?v=dX1EFuQJEwI&list=PLkCFuq1QH7nlaA9b8ZkHsJCiuGhHdkyl1*